

Advanced CNN Architectures

CS480/680 - University of Waterloo

Part 2: VGG and ResNet

Lecture 12 - Continued

Contents

1	Introduction: The Architecture Design Question	3
2	VGG Architecture (2014)	3
2.1	Core Principle: Small Stacked Filters	3
2.2	Motivation	3
2.3	VGG Solution: Stack of 3×3 Filters	4
2.4	Mathematical Analysis: Receptive Field	4
2.4.1	Single Large Filter	4
2.4.2	Stack of Three 3×3 Filters	5
2.5	Additional Benefits of Stacked Small Filters	5
2.6	VGG Design Rules	6
3	ResNet: Residual Networks (2015)	6
3.1	The Depth Problem	6
3.2	Key Insight: Residual Connections	6
3.3	Residual Block Architecture	7
3.4	Mathematical Formulation	7
3.5	Detailed Residual Block	8
3.6	Complete ResNet Architecture	8
3.7	Why Residual Connections Work	8
3.7.1	1. Gradient Flow	8
3.7.2	2. Effective Depth	9
3.7.3	3. Ensemble Effect	9
3.8	ResNet Variants	9
4	Batch Normalization	10
4.1	Motivation	10
4.2	Batch Normalization Algorithm	10
4.3	Why Batch Normalization Helps	10
5	Training Deep Networks: Summary	11
6	Applications Beyond Computer Vision	12
6.1	General Applicability	12
6.2	Application Domains	12
6.2.1	1D CNNs	12
6.2.2	2D CNNs	12
6.2.3	3D CNNs	13
6.3	Tensor Operations	13

7	Design Principles Summary	14
8	Historical Impact	15
8.1	ImageNet Competition Results	15
9	Conclusion	15
9.1	Key Takeaways	15
9.2	Modern Architecture Trends	16

1 Introduction: The Architecture Design Question

The Million Dollar Question

When designing a neural network, what is the right architecture?

This is one of the most important questions in deep learning, and much of the work in both academia and industry focuses on designing better architectures. Architecture design is:

- More of an **art than a science**
- **Problem-dependent** - no universal solution
- **Non-obvious** - what works for one task may not work for another

Despite the challenges, there are proven **rules of thumb** and **design principles** that consistently produce good results. Two major breakthroughs in CNN architecture design are:

1. **VGG (2014):** Visual Geometry Group, Oxford - Stack small filters instead of large filters
2. **ResNet (2015):** Microsoft Research - Residual connections enable very deep networks

2 VGG Architecture (2014)

2.1 Core Principle: Small Stacked Filters

VGG Key Insight

Replace single large filters with stacks of smaller filters

Benefits:

- Fewer parameters
- Deeper network
- More nonlinear transformations
- Better feature learning

2.2 Motivation

In early CNNs, researchers used large filters:

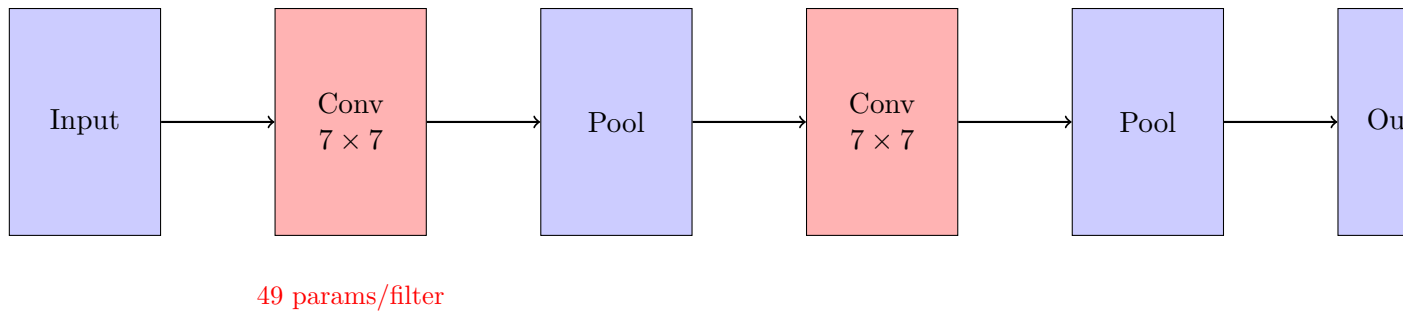
- 11×11 filters (AlexNet)
- 7×7 filters
- 5×5 filters

Intuition: Larger filters can capture more complex features in a single layer.

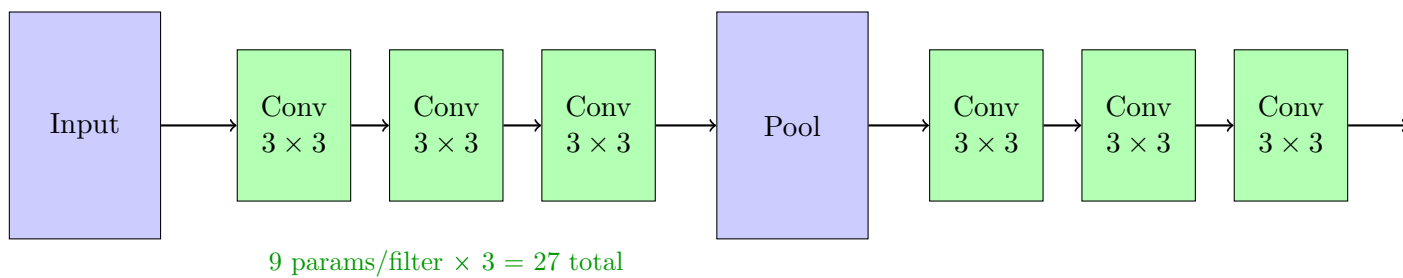
Problem: Large filters require many parameters and don't enable deep architectures.

2.3 VGG Solution: Stack of 3×3 Filters

Traditional: Large Filter



VGG: Stacked Small Filters



2.4 Mathematical Analysis: Receptive Field

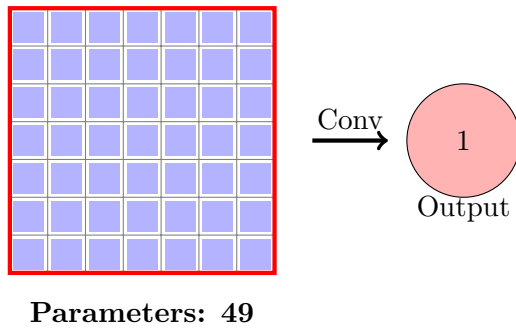
Question: How does a stack of 3×3 filters compare to a single 7×7 filter?

2.4.1 Single Large Filter

A 7×7 convolutional filter:

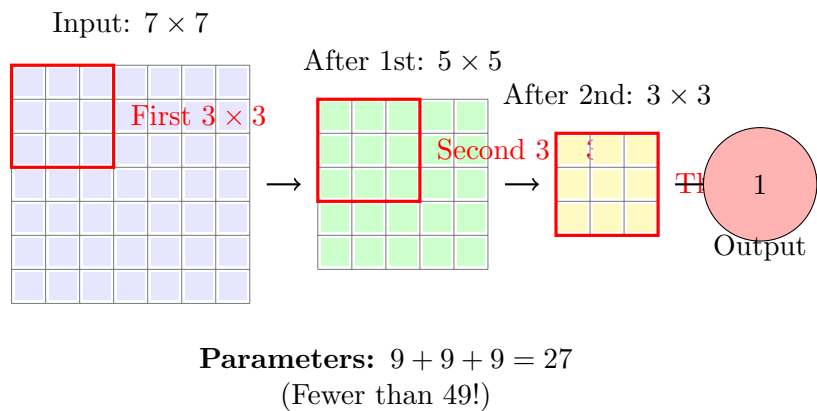
- **Input region:** $7 \times 7 = 49$ pixels
- **Output:** 1 feature value
- **Parameters per filter:** 49

7×7 receptive field



2.4.2 Stack of Three 3×3 Filters

Three consecutive 3×3 filters:



Effective receptive field: Same as 7×7 (covers same input region)

Parameters comparison:

- Single 7×7 filter: **49 parameters**
- Three 3×3 filters: $9 + 9 + 9 = 27$ **parameters**
- **Reduction:** $27/49 \approx 55\%$ **fewer parameters!**

2.5 Additional Benefits of Stacked Small Filters

1. More Nonlinear Activations

- Single 7×7 : 1 activation function applied
- Three 3×3 : 3 activation functions applied
- More nonlinearity = more expressive power

2. Deeper Network

- Depth helps learn hierarchical features
- Each layer builds on previous representations

3. Better Optimization

- Smaller filters easier to optimize
- More gradual feature transformation

2.6 VGG Design Rules

VGG Architecture Guidelines

1. Use **only** 3×3 **filters** (occasionally 1×1)
2. **Stack multiple convolutional layers** before pooling
3. **Double the number of filters** after each pooling layer
4. Use **same padding** to maintain spatial dimensions
5. **Apply max pooling** with 2×2 windows, stride 2

3 ResNet: Residual Networks (2015)

3.1 The Depth Problem

Challenge: Training Very Deep Networks

Problem: As network depth increases, performance degrades:

- Gradient vanishing/exploding becomes worse
- Even with ReLU activations and careful initialization
- Training error increases (not just overfitting!)
- Optimization becomes extremely difficult

Question: Can we design architectures that benefit from depth without degradation?

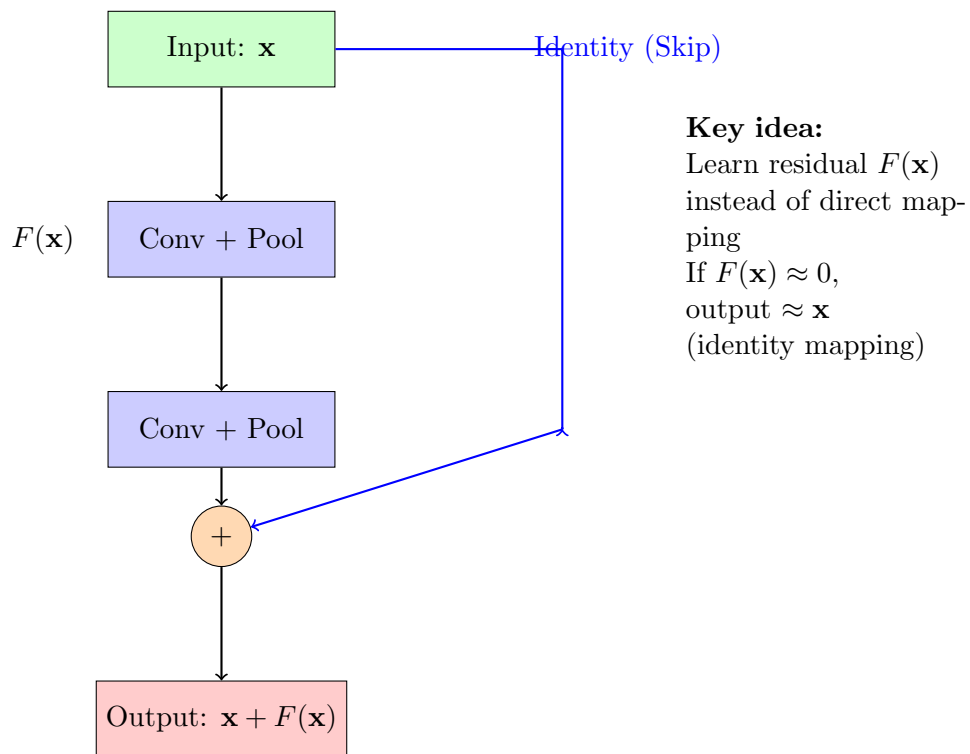
3.2 Key Insight: Residual Connections

Observation: If we add layers to a network, in the worst case, those layers should learn to do nothing (identity mapping). Then the deeper network would perform at least as well as the shallower one.

Problem: Learning identity mapping is hard for standard layers.

Solution: Make identity mapping the default through skip connections!

3.3 Residual Block Architecture



3.4 Mathematical Formulation

Traditional layer:

$$\mathbf{y} = F(\mathbf{x}, \{W_i\}) \quad (1)$$

Residual block:

$$\mathbf{y} = F(\mathbf{x}, \{W_i\}) + \mathbf{x} \quad (2)$$

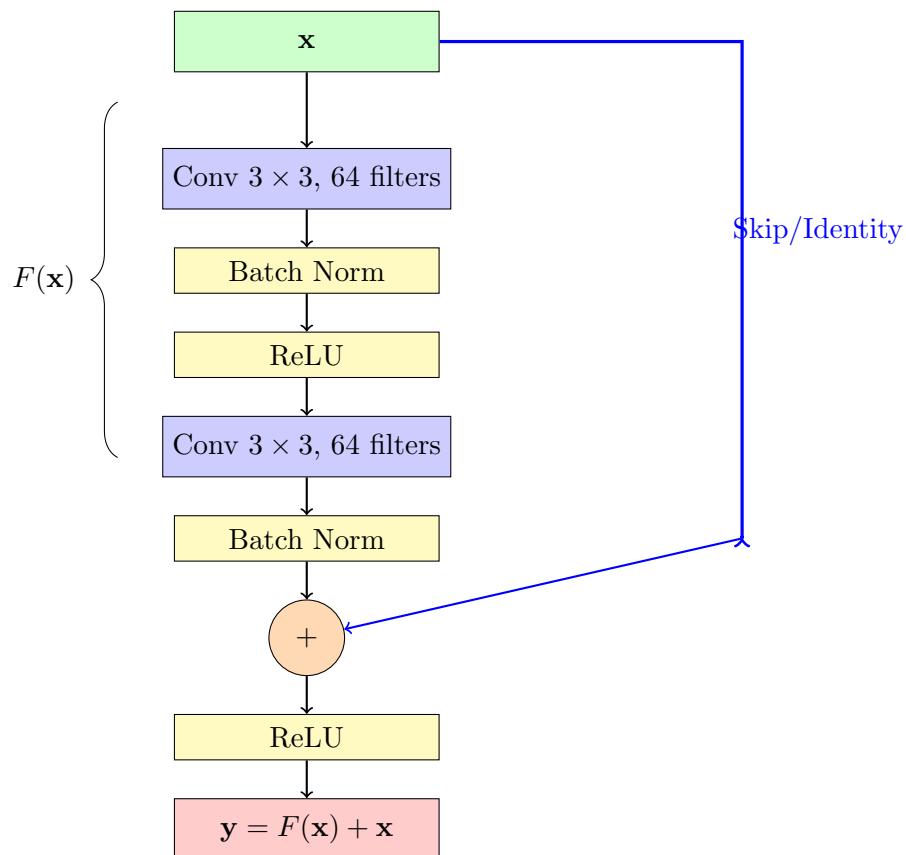
where:

- $\mathbf{x}$ is the input
- $F(\mathbf{x}, \{W_i\})$ is the residual mapping learned by the layers
- $\{W_i\}$ are the weights of the layers in the block
- $+$ is element-wise addition

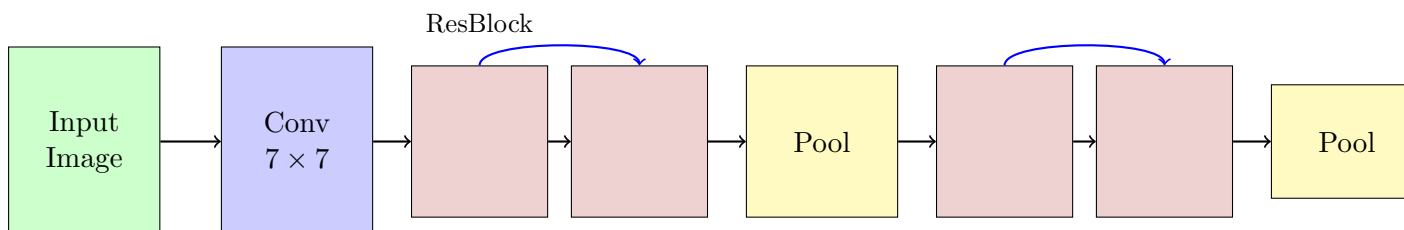
Why it works:

- If layers are initialized near zero: $F(\mathbf{x}) \approx 0 \Rightarrow \mathbf{y} \approx \mathbf{x}$ (identity)
- Network can choose to use or ignore additional layers
- Easier to optimize residual $F(\mathbf{x})$ than learn full transformation

3.5 Detailed Residual Block



3.6 Complete ResNet Architecture



ResNet can have 50, 101, 152, or even 1000+ layers!
Skip connections enable training of very deep networks

3.7 Why Residual Connections Work

3.7.1 1. Gradient Flow

Without skip connections:

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \cdot \frac{\partial F(\mathbf{x})}{\partial \mathbf{x}} \quad (3)$$

Gradient must flow through all layers, prone to vanishing.

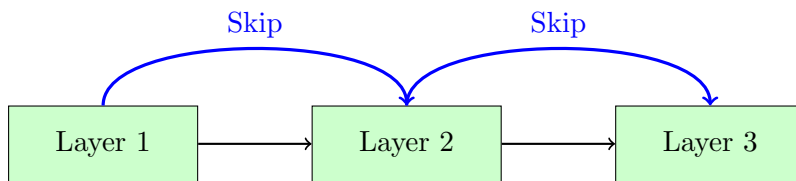
With skip connections:

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \left(\frac{\partial F(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I} \right) \quad (4)$$

where $\mathbf{I}$ is the identity. Gradient always has a direct path (the $+\mathbf{I}$ term).



Long path: gradient vanishes



Short paths: gradient preserved

3.7.2 2. Effective Depth

Consider a ResNet with 100 layers and skip connections every 2 layers:

- **Physical depth:** 100 layers
- **Effective depth along skip connections:** ~ 50 layers
- Gradients can flow through multiple paths of varying lengths
- Shorter paths dominate gradient propagation

3.7.3 3. Ensemble Effect

ResNets can be viewed as an ensemble of many paths:

- A ResNet with n skip connections contains 2^n different paths
- Each path has different effective depth
- Training implicitly creates an ensemble of networks

3.8 ResNet Variants

1. **ResNet-50:** 50 layers, widely used baseline
2. **ResNet-101:** 101 layers
3. **ResNet-152:** 152 layers, won ImageNet 2015
4. **ResNet-1000:** 1000+ layers, demonstrates extreme depth capability

4 Batch Normalization

4.1 Motivation

During training, internal activations can shift in distribution (internal covariate shift):

- Values may become very large or very small
- Makes optimization difficult
- Exacerbates gradient vanishing/exploding

4.2 Batch Normalization Algorithm

For a mini-batch of activations $\{x_1, x_2, \dots, x_m\}$:

1. **Compute batch statistics:**

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad (5)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (6)$$

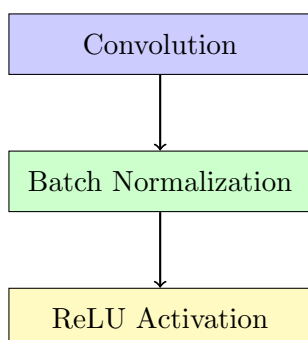
2. **Normalize:**

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (7)$$

3. **Scale and shift (learnable parameters):**

$$y_i = \gamma \hat{x}_i + \beta \quad (8)$$

where γ and β are learned parameters, ϵ is a small constant for numerical stability.



Benefits:

- Stabilizes training
- Allows higher learning rates
- Reduces sensitivity to initialization
- Acts as regularizer

4.3 Why Batch Normalization Helps

1. **Reduces internal covariate shift**

- Keeps activations in reasonable range
- Prevents extreme values

2. **Smooths optimization landscape**

- Makes loss surface more Lipschitz
- Gradients more predictable

3. Enables higher learning rates

- Normalized activations prevent divergence
- Faster convergence

5 Training Deep Networks: Summary

Techniques for Training Deep Networks

1. Architecture Design:

- Use small filters (3×3) stacked deeply (VGG)
- Add skip connections for very deep networks (ResNet)
- Consider bottleneck designs for efficiency

2. Normalization:

- Batch normalization between layers
- Layer normalization for RNNs
- Group normalization for small batches

3. Regularization:

- Dropout
- L2 weight decay
- Data augmentation

4. Optimization:

- Adam optimizer (adaptive learning rates)
- Learning rate schedules
- Gradient clipping

5. Initialization:

- Xavier/Glorot initialization
- He initialization (for ReLU)
- Orthogonal initialization

6 Applications Beyond Computer Vision

6.1 General Applicability

CNNs for Sequential and Spatial Data

CNNs are useful whenever data has:

- **Local patterns:** Features are local rather than global
- **Translation invariance:** Same pattern can appear at different positions
- **Hierarchical structure:** Can build complex features from simple ones

6.2 Application Domains

6.2.1 1D CNNs

Natural Language Processing:

- Text classification
- Sentiment analysis
- Named entity recognition
- Machine translation

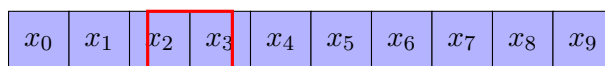
Time Series Analysis:

- Financial forecasting
- Signal processing
- Sensor data analysis

Audio Processing:

- Speech recognition
- Music generation
- Audio classification

1D Sequence (text, audio, time series)



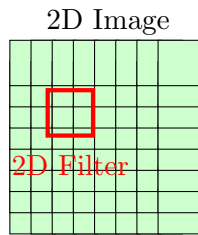
1D Filter

6.2.2 2D CNNs

Computer Vision:

- Image classification
- Object detection
- Semantic segmentation

- Face recognition
- Medical image analysis
- Autonomous driving



6.2.3 3D CNNs

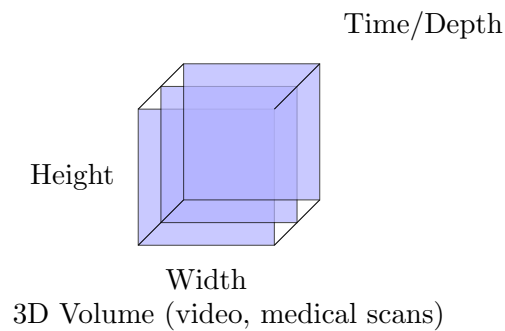
Video Analysis:

- Action recognition
- Video classification
- Temporal segmentation

Medical Imaging:

- 3D CT/MRI analysis
- Volumetric segmentation
- Organ detection

Dimensions: Width $\times$ Height $\times$ Time (or Depth)



6.3 Tensor Operations

Definition 1 (Tensor). *A tensor is a multi-dimensional array that generalizes:*

- *Scalar: 0D tensor*
- *Vector: 1D tensor*
- *Matrix: 2D tensor*
- *3D array: 3D tensor*
- *N-dimensional array: N-D tensor*

TensorFlow naming: The framework is named after the concept of tensor operations flowing through computational graphs.

7 Design Principles Summary

Modern CNN Architecture Best Practices

1. Filter Size:

- Prefer 3×3 filters (occasionally 1×1)
- Stack multiple small filters instead of large ones
- Effective receptive field grows with depth

2. Depth:

- Deeper is generally better (with proper techniques)
- Use residual connections for very deep networks (50+ layers)
- Add batch normalization between layers

3. Width (Number of Filters):

- Start with 32-64 filters in early layers
- Double filters after each pooling/stride operation
- Modern networks: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$

4. Skip Connections:

- Add skip connection every 2-3 layers
- Essential for networks deeper than 20-30 layers
- Enables gradient flow and faster convergence

5. Normalization:

- Batch normalization after conv, before activation
- Stabilizes training and allows higher learning rates
- Acts as implicit regularization

6. Activation Functions:

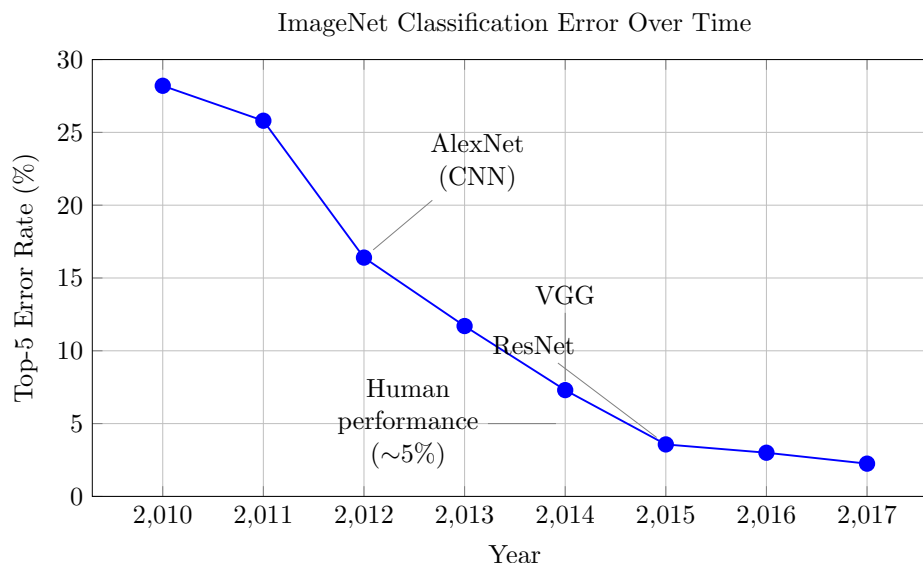
- ReLU is standard choice
- Consider Leaky ReLU or PReLU for very deep networks
- Apply after normalization

7. Pooling:

- Max pooling: 2×2 with stride 2
- Alternative: strided convolutions (no separate pooling)
- Global average pooling before final classification layer

8 Historical Impact

8.1 ImageNet Competition Results



Key Milestones:

- **2012 - AlexNet:** First deep CNN, 16.4% error (vs 25.8% previous year)
- **2014 - VGG:** Small stacked filters, 7.3% error
- **2015 - ResNet:** Residual connections, 3.57% error (surpassed human performance)
- **2015+:** Error continues to decrease with architectural innovations

9 Conclusion

9.1 Key Takeaways

1. **Architecture matters:** Design choices significantly impact performance
2. **Small filters work better:** Stack 3×3 instead of using large filters
 - Fewer parameters
 - More nonlinearity
 - Better optimization
3. **Depth is powerful but challenging:** Deeper networks learn better representations, but require:
 - Residual connections (skip connections)
 - Batch normalization
 - Careful initialization
4. **Skip connections enable extreme depth:** ResNet demonstrated networks with 100+ layers
 - Shorter gradient paths
 - Option to ignore unhelpful layers

- Ensemble of paths
5. **CNNs generalize beyond vision:** Applicable to any data with spatial/sequential structure
 6. **Empirical design process:** Architecture design remains partially an art
 - Follow established principles
 - Experiment and iterate
 - Validate on task-specific data

9.2 Modern Architecture Trends

Current research explores:

- **Neural Architecture Search (NAS):** Automatically discover architectures
- **Efficient networks:** MobileNet, EfficientNet for resource-constrained devices
- **Attention mechanisms:** Self-attention and transformers (going beyond convolution)
- **Hybrid architectures:** Combining CNNs with other architectures

*These notes are based on Lecture 12, Part 2 from CS480/680 at the University of Waterloo
Architecture design is an art informed by science - experiment, learn, and iterate!*